CSC485 A1

Shilin Zhang

September 2024

1 Question 1

1.1 a

Step	Stack	Buffer	New dep	Transition
0	[ROOT]	[To, raise, those, doubts, is, to, resolve, them]		
1	[ROOT, To]	[raise, those, doubts, is, to, resolve, them]		SHIFT
2	[ROOT, To, raise]	[those, doubts, is, to, resolve, them]		SHIFT
3	[ROOT, raise]	[those, doubts, is, to, resolve, them]	$\text{raise} \xrightarrow{\text{mark}} \text{To}$	LEFT-ARC
4	[ROOT, raise, those]	[doubts, is, to, resolve, them]		SHIFT
5	[ROOT, raise, those, doubts]	[is, to, resolve, them]		SHIFT
6	[ROOT, raise, doubts]	[is, to, resolve, them]	$\text{doubts} \xrightarrow{\text{det}} \text{those}$	LEFT-ARC
7	[ROOT, raise]	[is, to, resolve, them]	$\text{raise} \xrightarrow{\text{dobj}} \text{doubts}$	RIGHT-ARC
8	[ROOT, raise, is]	[to, resolve, them]		SHIFT
9	[ROOT, is]	[to, resolve, them]	$\text{is} \xrightarrow{\text{csubj}} \text{raise}$	LEFT-ARC
10	[ROOT, is, to]	[resolve, them]		SHIFT
11	[ROOT, is, to, resolve]	[them]		SHIFT
12	[ROOT, is, resolve]	[them]	$\text{resolve} \xrightarrow{\text{mark}} \text{to}$	LEFT-ARC
13	[ROOT, is, resolve, them]	[]		SHIFT
14	[ROOT, is, resolve]	[]	$\text{resolve} \xrightarrow{\text{dobj}} \text{them}$	RIGHT-ARC
15	[ROOT, is]	[]	$\text{is} \xrightarrow{\text{xcomp}} \text{resolve}$	RIGHT-ARC
16	[ROOT]	[]	$\text{ROOT} \xrightarrow{\text{root}} \text{is}$	RIGHT-ARC

1.2 b

If we don't count ROOT when we count the words, then we have exactly 2n steps to parse the sentence.
For example, in the example that we are doing in part a, it has 8 words and we take 16 steps to finish the parsing.
The reason why we need exactly 2n steps is because we know that every word need to go from buffer to stack exactly once and each word need to be moved out of stack exactly once (with LEFT-ARC or RIGHT-ARC) so we will have exactly 2 steps per word so we have 2n steps overall.

1.3 g

perform dependency parsing for "To ask those questions is to answer them."

Dependency parsing involves analyzing the grammatical structure of a sentence to understand the relationships between words. Here's a breakdown of the sentence **"To ask those questions is to answer them."** using dependency parsing:

1. **"To ask"** - This is the main verb phrase that introduces the action.
   - **"ask"** - Root verb.
   - **"To"** - Infinitive marker (auxiliary).
2. **"those questions"** - This noun phrase serves as the object of the verb "ask."
   - **"questions"** - Head noun.
   - **"those"** - Determiner (modifier).
3. **"is"** - The main verb of the sentence, linking the subject to the complement.
4. **"to answer them"** - This is an infinitive phrase that acts as a subject complement.
   - **"answer"** - Root verb of the infinitive phrase.
   - **"to"** - Infinitive marker.
   - **"them"** - Pronoun acting as the object of the verb "answer."

Dependency Relations

- **"ask"** → **"To"** (infinitive marker)
- **"questions"** → **"ask"** (direct object)
- **"those"** → **"questions"** (determiner)
- **"is"** → **"ask"** (subject)
- **"answer"** → **"is"** (subject complement)
- **"to"** → **"answer"** (infinitive marker)
- **"them"** → **"answer"** (object)

Visual Representation

In a tree structure, it might look like this:

vbnet Copy code

```vbnet
        is
        |
      ask
       / \
    to     answer
    |        |
    those   them
    |
questions
```

This illustrates the relationships among the words, with "is" as the main verb linking "ask" and "to answer them."

It seems like that chatGPT is a zero-shot dependency parser where there is no training needed in order for it to produce a valid and reasonable parse.
After comparing the parse that my model give with the parse that chatGPT give, I found some advantages and disadvantages for both system.
Advantages of chatGPT:
chatGPT is interactive where after dependency parsing, it can allow user to ask follow-up questions or clarify anywhere that is not clear inside the parse step which our model cannot.
chatGPT is more flexible with its response compared to my model where my model follows a fixed format to response (like not human readable sentence) but chatGPT can use actual sentences to explain the parse result which will be more user-friendly.
Disadvantages of chatGPT:
chatGPT's response does not follow strictly with the dependency parsing framework so the result of parsing make not be as precise as our model in some situations.
Similarly, chatGPT does not implement an explicit parsing algorithm meaning that the acurracy of the parse may not always be really high across different sentences.
Advantages of My Model:
Since our model is trained based off of a database and a well-defined algorithm, our model produce more precise and consistent results.
Our model is also more standardized since it is trained from an algorithm so the precision of the parse is better under a situation where the parse needs to follow this specific algorithm.
Disadvantages of My Model:
Our model is really time-costly where we need to train the model on the database and validate and test it because anyone can use it.
And if we have a really long and complex sentence, then our algorithm might take really long to parse it following the algorithm.
The performance of our model may also rely on the quality of the dataset that we train on for example if the dataset does not have idioms, then our model will have really bad accuracy for the parse.

2 Question 2

2.1 a

The reason why the above parsing mechanism is not sufficient with non-projective tree is that the the algorithm above only consider the last 2 words in the stack when choosing left-arc, right-arc or shift and there is no way to go back to a word that has been removed from the stack by a left-arc or right-arc.
If we have a dependency d_1 that cross another dependency d_2, then one word from d_1 will be in between d_2 meaning that we must remove the word between d_2 in order for us to parse the dependency d_1 since the mechanism only considers the last two words from the stack.
In order to remove this word in between, we need either a left-arc or right-arc but we will not have the two word that are involved in the dependency as the last 2 words in the stack since there is a crossing dependency so the mechanism will get stuck on crossing dependency since it is not sufficient to only look at the last 2 words from the stack.

2.2 b

```
● wolf:~/CSC485_A1$ python3 run_test.py q2
  Reading data... Done.
  100%|                                                  | 12543/12543 [00:00<00:00, 847755.37it/s]
  train: 12081/12543 (96.3%)
  100%|                                                  | 2001/2001 [00:00<00:00, 1015340.23it/s]
  dev: 1957/2001 (97.8%)
  100%|                                                  | 2077/2077 [00:00<00:00, 2360219.29it/s]
  test: 2036/2077 (98.0%)
```

2.3 c

Tree 1:
We first find the gap degree of every word in the sentence.

word	descendants	contiguous substrings	number of contiguous substrings	gap degree
ROOT	[To, raise, those, doubts, is, to, resolve, them]	[To raise those doubts is to resolve them]	1	0
To	[]	[To]	1	0
raise	[To, doubts, those]	[To raise those doubts]	1	0
those	[]	[those]	1	0
doubts	[those]	[those doubts]	1	0
is	[raise, To, doubts, those, resolve, to, them]	[To raise those doubts is to resolve them]	1	0
to	[]	[to]	1	0
resolve	[to, them]	[to resolve them]	1	0
them	[]	[them]	1	0

We see that every word in the dependency tree has gap degree of 0 so the gap degree of the dependency tree is 0.
Tree 2:
We still find the gap degree of every word in the sentence.

word	descendants	contiguous substrings	number of contiguous substrings	gap degree
ROOT	[Sam, met, a student, today, who, has, a linguistics degree]	[Sam met a student today who has a linguistics degree]	1	0
Sam	[]	[Sam]	1	0
met	[Sam, a student, has, who, a linguistics degree, today]	[Sam met a student today who has a linguistics degree]	1	0
a student	[has, who, a linguistics degree]	[a student, who has a linguistics degree]	2	1
today	[]	[today]	1	0
who	[]	[who]	1	0
has	[who, a linguistics degree]	[who has a linguistics degree]	1	0
a linguistics degree	[]	[a linguistics degree]	1	0

We see that the gap degree of the word "a student" is 1 because the word "today" breaks the word "a student" and its descendants into 2 contiguous substrings so the gap degree of this dependency tree is 1.

2.4 e

We know from wikipedia that the uniform distribution has $\mu = \frac{1}{2}(a+b)$ and $\sigma^2 = \frac{1}{12}(b-a)^2$.
From μ:

$$\begin{aligned} \frac{1}{2}(a+b) &= 0 \\ a+b &= 0 \\ a &= -b \end{aligned} \tag{1}$$

Then substitute it into variance, we get:

$$\begin{aligned} \frac{1}{12}(b-(-b))^2 &= \frac{2}{m} \\ \frac{1}{12}(4b^2) &= \frac{2}{m} \\ \frac{1}{3}b^2 &= \frac{2}{m} \\ b^2 &= \frac{6}{m} \\ b &= \pm\sqrt{\frac{6}{m}} \end{aligned} \tag{2}$$

Since b is upper bound, then we choose b to be positive root and then we can get a from $a = -b$ meaning that we have $b = \sqrt{\frac{6}{m}}$ as the upper bound and $a = -\sqrt{\frac{6}{m}}$ as the lower bound.

2.5 g

We cannot have a terms that only includes D_A but not H_A because in arc scoring, we are trying to determine whether there exist a relationship between any possible heads and any possible dependencies so we need to examine the relationship between H_A and D_A and we add a bias terms based on the head H_A.
If we add a term that just include D_A but not H_A, this means that we are assigning a score only base on its potential

as a dependency, ignoring other words that it might depend on which is not suitable for determining whether a relationship exist or not.
On the other hand, in label scoring, we are trying to determine the type of an existing relationship so it is better to get the score of H_A and D_A separately since they could provide independent information about the relationship that we are investigating.

2.6 h

In a standard classification problem, we multiply the input by a weight matrix and add a per-class bias to produce a class score for the given input because we want to compute linear combination of the input features for each class and use a softmax function or some similar function to get the probabilities of each class.
But in the context of dependency parsing, we want to predict the syntactic relationship within a sentence where we need to consider the interaction between every word in the sentence to come up with pairs of heads and dependencies. To accomplish this task, we need Biaffine transformation to first, get weighted features for each input (D_A, H_A, etc) and second, consider pairwise interactions between all features of dependencies and heads and calculate scores. This is where the multiplication twice comes in.

2.7 i

Constraint 1: No Self Loop
The first constraint that is enforced is the no self loop constraint where a word cannot be the head of itself. This is enforced by setting the diagonal entries to False.
Constraint 2: The root vertex cannot be a dependent
The second constraint that is enforced is that the root vertex can only be head of some word but can never be dependent of any word. This is enforced by setting all entries with root or 0 as the head to False.
Constraint 3: The root dependency relation can only be used when root is head
The third constraint that is enforced is that the root dependency relation which has label 0 can only appear if the root vertex is the head. This is enforced by setting all entries where the head is not 0 but has a dependency relation label of 0 to False.

2.8 j

The reason why we need a MST over just simple argmax is because all the constraints for dependency parsing.
In dependency parsing, the dependency tree must satisfy the constraint that it is acyclic, connected, single rooted and every word have exactly one head which can all be guaranteed with a MST.
If we use argmax without any regulations for dependency parsing, argmax may violate these constraints such as producing cycles or disconnected graphs or multiple heads for a single word so it is not really a good choice for making final arc prediction.
But on the other hand, when we are finding labels, we could use argmax since the arcs are already there and labels don't have the constraints that arcs do.

2.9 l

2.9.1 Sentence 1: "I'm not fond of the Google-hates-privacy argument"

.
Transition-based Parsing:

```
1   I       _   _   _   _   4   nsubj     _   _
2   'm      _   _   _   _   4   cop       _   _
3   not     _   _   _   _   4   advmod    _   _
4   fond    _   _   _   _   0   root      _   _
5   of      _   _   _   _   12  case      _   _
6   the     _   _   _   _   12  det       _   _
7   Google  _   _   _   _   9   obl:npmod  _   _
8   -       _   _   _   _   9   punct     _   _
9   hates   _   _   _   _   11  amod      _   _
10  -       _   _   _   _   11  punct     _   _
11  privacy _   _   _   _   12  compound  _   _
12  argument _  _   _   _   4   obl       _   _
```

Graph-based Parsing:

```
1   I       _   _   _   _   4   nsubj     _   _
2   'm      _   _   _   _   4   cop       _   _
3   not     _   _   _   _   4   advmod    _   _
4   fond    _   _   _   _   0   root      _   _
5   of      _   _   _   _   12  case      _   _
6   the     _   _   _   _   12  det       _   _
7   Google  _   _   _   _   9   nsubj     _   _
8   -       _   _   _   _   9   punct     _   _
9   hates   _   _   _   _   4   advcl     _   _
10  -       _   _   _   _   11  punct     _   _
11  privacy _   _   _   _   9   obj       _   _
12  argument _  _   _   _   4   punct     _   _
```

Correct Parse From Corpus:

```
# sent_id = weblog-blogspot.com_marketview_20050224181500_ENG_20050224_181500-0002
# text = I'm not fond of the Google-hates-privacy argument
1-2 I'm _   _   _   _   _   _   _   _
1   I   I   PRON    PRP Case=Nom|Number=Sing|Person=1|PronType=Prs  4   nsubj   4:nsubj _
2   'm  be  AUX VBP Mood=Ind|Tense=Pres|VerbForm=Fin    4   cop 4:cop   _
3   not not PART    RB  _   4   advmod  4:advmod    _
4   fond    fond    ADJ JJ  Degree=Pos  0   root    0:root  _
5   of  of  ADP IN  _   12  case    12:case _
6   the the DET DT  Definite=Def|PronType=Art   12  det 12:det  _
7   Google  Google  PROPN   NNP Number=Sing 9   nsubj   9:nsubj SpaceAfter=No
8   -   -   PUNCT   HYPH    _   9   punct   9:punct SpaceAfter=No
9   hates   hate    VERB    VBZ Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin   12  compound    12:compound SpaceAfter=No
10  -   -   PUNCT   HYPH    _   9   punct   9:punct SpaceAfter=No
11  privacy privacy NOUN    NN  Number=Sing 9   obj 9:obj   _
12  argument    argument    NOUN    NN  Number=Sing 4   obl 4:obl:of    _
```

Preferred Parse:

In the both parses, they fail to recognize the word "hates" is describing the word "argument".
Specifically, in the first parse, it attached the word "hates" to the word "privacy" but in the second parse, it attached the word "hates" to the word "fond".
The first parse also incorrectly attached the word "privacy" to the word argument.
Overall, for the 2 parses, I prefer the second parse because it correctly find the head of the word "privacy" as the word "hates" unlike the first parse which attached the word "privacy" to the word "argument" which leads to completely different meaning.

2.9.2 Sentence 2: "BREYER filed a concurring opinion, in which KENNEDY, SOUTER, and GINSBURG joined."

Transition-based Parsing:

```
1   BREYER  _   _   _   _   2   nsubj   _   _
2   filed   _   _   _   _   0   root    _   _
3   a   _   _   _   _   5   det _   _
4   concurring  _   _   _   _   5   amod    _   _
5   opinion _   _   _   _   2   obj _   _
6   ,   _   _   _   _   5   punct   _   _
7   in  _   _   _   _   9   case    _   _
8   which   _   _   _   _   9   nmod:poss   _   _
9   KENNEDY _   _   _   _   5   nmod    _   _
10  ,   _   _   _   _   11  punct   _   _
11  SOUTER  _   _   _   _   9   conj    _   _
12  ,   _   _   _   _   15  punct   _   _
13  and _   _   _   _   15  cc  _   _
14  GINSBURG    _   _   _   _   15  nsubj   _   _
15  joined  _   _   _   _   9   conj    _   _
16  .   _   _   _   _   2   punct   _   _
```

Graph-based Parsing:

```
1   BREYER  _   _   _   _   2   nsubj   _   _
2   filed   _   _   _   _   0   root    _   _
3   a   _   _   _   _   5   det _   _
4   concurring  _   _   _   _   5   amod    _   _
5   opinion _   _   _   _   2   obj _   _
6   ,   _   _   _   _   2   punct   _   _
7   in  _   _   _   _   8   case    _   _
8   which   _   _   _   _   15  obl _   _
9   KENNEDY _   _   _   _   2   nsubj   _   _
10  ,   _   _   _   _   11  punct   _   _
11  SOUTER  _   _   _   _   15  nsubj   _   _
12  ,   _   _   _   _   14  punct   _   _
13  and _   _   _   _   14  cc  _   _
14  GINSBURG    _   _   _   _   11  conj    _   _
15  joined  _   _   _   _   2   advcl   _   _
16  .   _   _   _   _   2   punct   _   _
```

Correct Parse From Corpus:

```
# sent_id = weblog-blogspot.com_aggressivevoicedaily_20060629164800_ENG_20060629_164800-0004
# text = BREYER filed a concurring opinion, in which KENNEDY, SOUTER, and GINSBURG joined.
1   BREYER  breyer  NOUN    NNS Number=Plur 2   nsubj   2:nsubj _
2   filed   file    VERB    VBD Mood=Ind|Tense=Past|VerbForm=Fin    0   root    0:root  _
3   a   a   DET DT  Definite=Ind|PronType=Art   5   det 5:det   _
4   concurring  concur  VERB    VBG VerbForm=Ger    5   amod    5:amod  _
5   opinion opinion NOUN    NN  Number=Sing 2   obj 2:obj|15:obl    SpaceAfter=No
6   ,   ,   PUNCT   ,   _   5   punct   5:punct _
7   in  in  ADP IN  _   8   case    8:case  _
8   which   which   PRON    WDT PronType=Int    15  obl 5:ref   _
9   KENNEDY KENNEDY PROPN   NNP Number=Sing 15  nsubj   15:nsubj    SpaceAfter=No
10  ,   ,   PUNCT   ,   _   11  punct   11:punct    _
11  SOUTER  SOUTER  PROPN   NNP Number=Sing 9   conj    9:conj:and|15:nsubj SpaceAfter=No
12  ,   ,   PUNCT   ,   _   14  punct   14:punct    _
13  and and CCONJ   CC  _   14  cc  14:cc   _
14  GINSBURG    GINSBURG    PROPN   NNP Number=Sing 9   conj    9:conj:and|15:nsubj _
15  joined  join    VERB    VBD Mood=Ind|Tense=Past|VerbForm=Fin    5   acl:relcl   5:acl:relcl SpaceAfter=No
16  .   .   PUNCT   .   _   2   punct   2:punct _
```

Preferred Parse:

The first parse thinks that the phrase "in which" is used to describe "KENNEDY" and thinks that "GINBURG" is used to describe "joined" and it failed to recognize that the word "joined" is actually describing the word "opinion" instead of the word "KENNEDY".

In the second parse, it failed to recognize that the word "KENNEDY" is actually describing the word "joined" and

it thinks that the word "GINSBURG" describes the word "SOUTER".
Overall, I prefer the second parse where it actually correctly identify the structure of the sentence around the "in which" phrase and realized that the verb after the in which phrase is describing something before the in which phrase where the first parse did not realize this structure and attached the second verb "joined" directly to "KENNEDY".

2.9.3 Sentence 3: "SCALIA filed a dissenting opinion, in which THOMAS and ALITO joined."

Transition-based Parsing:

```
1   SCALIA  _   _   _   _   2   nsubj   _   _
2   filed   _   _   _   _   0   root    _   _
3   a   _   _   _   _   5   det _   _
4   dissenting  _   _   _   _   5   amod    _   _
5   opinion _   _   _   _   2   obj _   _
6   ,   _   _   _   _   5   punct   _   _
7   in  _   _   _   _   12  mark    _   _
8   which   _   _   _   _   12  obj _   _
9   THOMAS  _   _   _   _   12  nsubj   _   _
10  and _   _   _   _   11  cc  _   _
11  ALITO   _   _   _   _   9   conj    _   _
12  joined  _   _   _   _   5   acl _   _
13  .   _   _   _   _   2   punct   _   _
```

Graph-based Parsing:

```
1   SCALIA  _   _   _   _   2   nsubj   _   _
2   filed   _   _   _   _   0   root    _   _
3   a   _   _   _   _   5   det _   _
4   dissenting  _   _   _   _   5   amod    _   _
5   opinion _   _   _   _   2   obj _   _
6   ,   _   _   _   _   2   punct   _   _
7   in  _   _   _   _   8   case    _   _
8   which   _   _   _   _   12  obl _   _
9   THOMAS  _   _   _   _   12  nsubj   _   _
10  and _   _   _   _   11  cc  _   _
11  ALITO   _   _   _   _   9   conj    _   _
12  joined  _   _   _   _   2   advcl   _   _
13  .   _   _   _   _   2   punct   _   _
```

Correct Parse From Corpus:

```
# sent_id = weblog-blogspot.com_aggressivevoicedaily_20060629164800_ENG_20060629_164800-0006
# text = SCALIA filed a dissenting opinion, in which THOMAS and ALITO joined.
1   SCALIA  SCALIA  PROPN   NNP Number=Sing 2   nsubj   2:nsubj _
2   filed   file    VERB    VBD Mood=Ind|Tense=Past|VerbForm=Fin    0   root    0:root  _
3   a   a   DET DT  Definite=Ind|PronType=Art   5   det 5:det   _
4   dissenting  dissent VERB    VBG VerbForm=Ger    5   amod    5:amod  _
5   opinion opinion NOUN    NN  Number=Sing 2   obj 2:obj|12:obl    SpaceAfter=No
6   ,   ,   PUNCT   ,   _   5   punct   5:punct _
7   in  in  ADP IN  _   8   case    8:case  _
8   which   which   PRON    WDT PronType=Int    12  obl 5:ref   _
9   THOMAS  THOMAS  PROPN   NNP Number=Sing 12  nsubj   12:nsubj    _
10  and and CCONJ   CC  _   11  cc  11:cc   _
11  ALITO   ALITO   PROPN   NNP Number=Sing 9   conj    9:conj:and|12:nsubj _
12  joined  join    VERB    VBD Mood=Ind|Tense=Past|VerbForm=Fin    5   acl:relcl   5:acl:relcl SpaceAfter=No
13  .   .   PUNCT   .   _   2   punct   2:punct _
```

Preferred Parse:

In the first parse, it successfully detected the second verb "joined" is used to describe the word "opinion" before the in which phrase however, it did not successfully parse the in which phrase where it says each of the 2 words is dependent of the word "joined" rathern than in describes which and which describe the word joined.

In the second parse, it successfully parsed the in which phrase, however, it failed to recognize that the second verb "joined" is actually describing opinion rather than the first verb "filed".

Overall, I prefer the first parse where even it fail to parse the in which structure, but it successfully identify that the head of "joined" is "opinion" not "filed" and the parse that it produce for the in which phrase is still within an understandable range where the parse is not too far from the truth (setting head of both words to joined rather than in depend on which and which depend on joined).